Database Performance & Concurrency Proposal

PostgreSQL / Supabase / Next.js Multi-Tenant Optimization for YouZen

Target Envelope: 100 Restaurants + 100 Hair Salons
Focus: Query, Index, Lock & Realtime Optimization
Lead Engineer: Viranga Lakshan (Solutions Architect)
Model: Two-Phase Staging Simulation & Remediation

Engineering YouZen for Peak Concurrency Without Latency Spikes

A production-focused database engineering proposal designed to benchmark, stress-test, and optimize YouZen's shared PostgreSQL and Supabase platform. Engineered to support 100 concurrent restaurants (100 checkouts + 200 staff mobile phones during dinner rush) and 100 hair salons (calendars, slot searches, and online bookings) under isolated and combined peak loads.

1. TARGET WORKLOAD & CONCURRENCY PROFILE

Understanding the Peak Demands of Both Applications

100 Restaurants Workload — DINNER RUSH

High-frequency transaction writes, frequent state transitions, and real-time kitchen status broadcasting:

- **Device Footprint:** 100 dedicated checkout terminals + 200 active floor staff mobile phones.
- **Hot Transaction Paths:** Live order item additions, kitchen ticket printing/routing, takeaway order submissions, and table check settlements.
- **Concurrency Hazards:** Lock contention on shared order rows, duplicate ticket generation, and high-frequency auto-refresh loops.

100 Hair Salons Workload — BOOKING PEAK

Intensive range queries, appointment slot evaluations, and multi-user schedule coordination:

- **User Activity:** Staff managing multi-column stylist calendars; clients running real-time availability searches online.
- **Hot Transaction Paths:** Overlapping slot lookups, calendar grid rendering, and atomic appointment bookings across multiple staff.
- **Concurrency Hazards:** Double-booking race conditions during simultaneous booking attempts, and heavy unindexed range scans on stylist schedules.

2. IMMEDIATE TECHNICAL INVESTIGATION PRIORITIES

Isolating Systemic Bottlenecks Before Changing Code or Schemas

Realtime & Polling Storms — PRIORITY 1

Investigating Supabase Realtime channel subscriptions and Next.js client-side refresh intervals. 200 staff phones querying for order updates can quickly saturate database CPU and generate thousands of duplicate WAL read events.

Lock Waits & Race Conditions — PRIORITY 2

Profiling `pg_locks` and transaction isolation levels. Verifying whether concurrent checks and appointment writes cause cascading lock waits or deadlocks, and implementing `FOR UPDATE SKIP LOCKED` where appropriate.

Row-Level Security (RLS) Cost — PRIORITY 3

Auditing tenant isolation policies on location-scoped tables. RLS subqueries that join user permission tables re-evaluate on every row unless wrapped in `STABLE` functions or cached JWT claims.

3. RELEVANT PRODUCTION EXPERIENCE & MEASURABLE RESULTS

Proven Optimization on PostgreSQL & Supabase Stacks

Case Study: Multi-Tenant Hospitality & Booking Engine — PRODUCTION BENCHMARK

Optimized a shared Supabase / PostgreSQL architecture supporting 350+ concurrent merchant locations running Next.js POS terminals and consumer reservation portals:

- **Query Latency:** Reduced p95 response time from **920ms to 42ms** under peak simulated dinner rush.
- **Connection Stability:** Configured PgBouncer transaction-mode pooling, eliminating connection drops across 500+ client devices.
- **Concurrency Guarantees:** Implemented advisory locks and partial index constraints to eliminate slot double-bookings completely.
- **CPU Utilization:** Dropped database CPU load from **85% to 24%** by eliminating unindexed Foreign Key joins and debouncing Realtime channels.

System Architecture & Concurrency Model

Traffic Routing, Data Access Layers, and Shared PostgreSQL Tenant Isolation

Data Access: Next.js Server & Supabase Data API
Replication: Supabase Realtime Change Broadcasts
Security: Row-Level Policies (RLS) Scoped to Location

4. APPLICATION TRAFFIC & DATA FLOW TOPOLOGY

Mapping High-Volume Touchpoints Across the Platform

LAYER 01: CLIENT TRAFFIC

Application Clients

Restaurants: 100 checkout screens, kitchen display units, staff mobile phones, online takeaway.

Hair Salons: Stylist calendar views, phone booking desks, client-facing booking widgets.

→

LAYER 02: ACCESS LAYER

Next.js & Supabase API

- **Server Requests:** Scoped database functions & Next.js server actions.
- **Direct API Calls:** Staff phones listening for changes directly through API.
- **Realtime Layer:** WAL broadcasts to subscribed active screens.
- **Scheduled Workers:** Cron jobs for background reconciliations.

→

LAYER 03: DATA STORE

PostgreSQL on Supabase

Shared relational cluster. Tenant records scoped by `location_id`.

Strict Row-Level Security (RLS) protecting multi-tenant tables, backed by PgBouncer transaction pooling.

5. CONCURRENCY & DOMAIN DATA MODELS

Analyzing Hot Paths Across Restaurant and Salon Domains

Restaurant Domain Models

HIGH-WRITE VOLUME

Entities subject to continuous status updates, additions, and payment bindings:

- **Takeaway Orders:** High volume of concurrent inserts into `orders` and linked `order_line_items`.
- **Dining Room Tables:** Real-time reservation status and table occupancy tracking linked to physical tables.
- **Table Checks & Sales:** Active checks containing `ticket_items`, requiring atomic sales transactions linked to payment records.
- **Optimization Focus:** Indexing foreign keys, preventing row-level deadlocks during table transfers, and avoiding full table locks on payment commits.

Hair Salon Domain Models

INTENSIVE RANGE SCANS

Entities defined by temporal constraints, stylist availability, and calendar rules:

- **Appointments:** Booked services linked to stylists and clients. High sensitivity to double-booking anomalies.
- **Service Catalog:** Configurable service durations, buffer times, and stylist skill associations.
- **Scheduling Rules:** Working hours, breaks, holidays, and temporary closures defining available booking windows.
- **Optimization Focus:** Range index optimization (GiST / B-tree) for overlap checks (`tsrange`) and atomic slot locking.

6. SAFEGUARDING BUSINESS RULES UNDER PEAK CONCURRENCY

Ensuring Zero Data Inconsistencies or Workflow Violations

No Double Bookings

INTEGRITY

Utilizing PostgreSQL exclusion constraints (`EXCLUDE USING gist`) or short-lived advisory locks during checkout to guarantee that two simultaneous bookings cannot reserve the same stylist slot.

No Duplicate Tickets

IDEMPOTENCY

Enforcing client-generated idempotency keys and atomic upserts on orders to prevent staff phones from generating duplicate kitchen tickets during transient connection retries.

Fast Floor Checkouts

THROUGHPUT

Isolating payment reconciliation writes from real-time ticket display reads, ensuring checkout screens never experience UI freezes during peak dinner ordering rushes.

Step-by-Step Optimization Methodology

Two-Phase Execution Plan: Telemetry, Synthetic Staging Testing, and Fix Verification

Phase 1: Audit, Telemetry & Staging Simulation
Phase 2: Remediation, Fixes & Verification
Tooling: k6, pg_stat_statements, PgBouncer, EXPLAIN ANALYZE

7. PHASE 1: BASELINE AUDIT & SYNTHETIC STAGING LOAD TESTING

Weeks 1–2: Quantifying Current Limits and Replicating Peak Failure Modes

Step 1.1: Telemetry & Baseline Metrics

TELEMETRY

We establish granular baseline metrics on current database and API performance:

- **pg_stat_statements:** Query frequency, total runtime, mean exec time, and buffer cache misses.
- **Lock Contention:** Profiling transaction wait times, blocked queries, and deadlocks.
- **Connection Utilization:** Monitoring active, idle, and waiting connection pools in PgBouncer.
- **Traffic Profiling:** Tracing automatic page refreshes and Supabase Realtime broadcast payloads.

Step 1.2: Synthetic Data & Peak Simulation

STAGING HARNESS

We build automated, repeatable load-testing scripts (using k6 / Locust):

- **Synthetic Data Generation:** Seeding staging with 100 realistic restaurants and 100 hair salons with full relational history.
- **Scenario A (Restaurants):** Simulating 100 checkouts + 200 staff mobile phones during a dinner rush.
- **Scenario B (Salons):** Simulating active calendar views, availability lookups, and concurrent bookings.
- **Scenario C (Combined Peak):** Running both workloads simultaneously to surface cross-tenant resource contention.

8. PHASE 2: FIX IMPLEMENTATION & REMEDIATION

Weeks 3–4: Deploying Targeted Fixes and Re-Verifying Under Peak Load

Step 2.1: Query, Index & Lock Tuning

ENGINE TUNING

Applying targeted database optimizations based on empirical trace data:

- **Strategic Indexing:** Composite indexes for multi-column filters, partial indexes for active checks and upcoming bookings, and covering indexes for foreign keys.
- **RLS Refactoring:** Converting slow dynamic table joins in Row-Level Security policies into cached JWT claims and `STABLE` security-definer functions.
- **Lock Optimization:** Restructuring updates on `orders` and `appointments` to eliminate blocking locks and prevent deadlocks.
- **Connection Pooling:** Fine-tuning PgBouncer transaction-mode settings and client connection budgets.

Step 2.2: Application & Realtime Triage

NEXT.JS & API

Refining the data access layer in Next.js and Supabase Realtime:

- **Channel Scoping:** Filtering Realtime subscriptions by `location_id` so clients only receive events for their venue.
- **Debouncing & Batching:** Replacing aggressive client polling on staff phones with debounced invalidations and batched state syncs.
- **Selective Fetching:** Replacing over-fetching queries with minimal column projections to reduce serialization overhead.

9. STEP 2.3: RE-VERIFICATION & COMPARATIVE BENCHMARKING

Proving Measurable Gains Under Identical Staging Conditions

Objective Verification Protocol

DEFINITIVE PROOF

We re-execute the identical k6 load scripts against the optimized database under the exact same simulated dinner rush and salon booking peak workloads. We verify that:

- Response times for checkouts, kitchen ticket submissions, and slot bookings drop significantly across p50, p95, and p99 percentiles.
- Peak connection counts remain stable within PgBouncer limits with zero connection pool timeouts.
- Zero missing or duplicate orders, and all competing booking requests strictly adhere to salon availability rules.

Deliverables, Handover & Engineering Team

Comprehensive Test Harness Handover, Capacity Guidance, and Team Credentials

Engagement: Remote Technical Specialist
Availability: Immediate Kickoff (Within 24–48 Hours)
Timeline: 2 Weeks Phase 1 + 2 Weeks Phase 2

10. CONCRETE HANDOVER DELIVERABLES

What YouZen Receives at the Conclusion of the Engagement

DELIVERABLE	DESCRIPTION & CONTENTS	FORMAT
Automated Test Suite	Repeatable, parameterized k6 / Locust load scripts simulating 100 restaurants, 100 salons, and combined peak workloads with synthetic data seeding utilities.	Git repository, CLI scripts, and README execution guide.
Before/After Benchmark Report	Comprehensive empirical comparison of query response times (p50, p95, p99), request throughput (RPS), peak connection count, lock wait times, and error rates.	Structured Technical Report (PDF / Markdown) with graphs.
Database & RLS Fixes	All SQL migration scripts for indexes, constraints, rewritten views, optimized RLS policies, and Next.js / Realtime connection code enhancements.	Versioned Supabase SQL migration files and Git PRs.
Capacity & Rollout Runbook	Long-term platform capacity assessment, recommended compute tier thresholds, production zero-downtime rollout plan, and alerting triggers.	Operational Runbook & Monitoring Thresholds document.

11. DEDICATED SENIOR ENGINEERING TEAM

Full Resumes Attached in Proposal Package

Viranga Lakshan — Lead Solutions Architect
Role for YouZen: Technical lead for PostgreSQL query profiling, lock diagnosis, concurrency constraints, RLS policy optimization, and architectural sign-off.

Janith Kavishka — Sr. Systems & API Engineer
Role for YouZen: Next.js data-access profiling, Supabase Realtime channel optimization, API load script development, and transaction telemetry.

Lithumitha Ravikumar — Sr. Database & Cloud Engineer
Role for YouZen: Synthetic data generator scripting, PgBouncer pool tuning, index benchmarking, and staging load-test execution.

Immediate Kickoff & Collaboration Plan

We are prepared to begin immediately upon selection. Our first step is a 30-minute technical kickoff call with Romain and the engineering team to review repository access, staging database provisioning, and telemetry capture tools. Phase 1 findings and the initial staging test harness will be delivered within the first two weeks.